Machine Learning — Lecture 1

Machine Learning Foundations and Generalization

FCIS — For Course Use Only

Dr. Fayçal Hamdi

Faculty of Computer and Information Systems

Lecture 1 at a Glance

FCIS — For Private Use Only

Machine Learning Foundations and Generalization

Course Learning Outcome

CLO1: Explain the fundamental concepts of machine learning, including learning paradigms, data representation, model assumptions, and evaluation principles.

Lecture purpose

- Establish a precise vocabulary for **learning from data**.
- Distinguish the major learning paradigms and prediction task types.
- Represent datasets, models, parameters, and learning objectives mathematically.
- Explain why **generalization to unseen data** is the central goal of machine learning.

Conceptual structure

Learning phase	Core question
1. Machine-learning foundations	What does it mean to learn from experience, and what learning setting do we have?
2. Representation and learning	How do we express examples, features, targets, models, and learning objectives?
3. Generalization and complexity	Why can a model fit observed data yet fail on new data?
4. Reliable evaluation	How do we select and evaluate models without contaminating the evidence?

FCIS - For Course Use Only

💡 Main idea: machine learning is not the goal of fitting the training data perfectly; it is the goal of learning **useful patterns that generalize beyond the training examples**.

Learning Objectives

FCIS — Course Use Only

By the end of Lecture 1, you will be able to:

1. Explain machine learning using the relationship between **task, experience, and performance**.
2. Distinguish supervised, unsupervised, and reinforcement learning.
3. Distinguish regression, classification, clustering, and sequential decision problems.
4. Represent a dataset using observations, features, targets, a feature matrix X, and a target vector y.
5. Distinguish model parameters from hyperparameters and explain the role of a loss function.
6. Explain model assumptions and why representation affects what can be learned.
7. Explain training, validation, testing, generalization, underfitting, and overfitting.
8. Identify data leakage and describe sound evaluation principles for unseen data.

For Course Use Only

Lecture Roadmap

FCIS — Course Use Only

From learning from experience to reliable generalization

Learning phase	Part	Core question
1. Machine-learning foundations	**I**	What does learning from experience mean, and is the problem supervised, unsupervised, or reinforcement learning?
2. Representation and learning	**II**	How do we represent data, targets, models, parameters, hyperparameters, and learning objectives?
3. Generalization and complexity	**III**	Why can a model fit training data well yet perform poorly on unseen data?
4. Reliable evaluation	**IV**	How do we select, validate, and test models without leakage or test-set contamination?

FCIS — For Course Use Only

💡 **Practice & consolidation:** connect the full vocabulary, diagnose learning behavior, identify common mistakes, and justify evaluation decisions.

Part I

FCIS — For Students Use Only

Machine Learning Foundations and Learning Settings

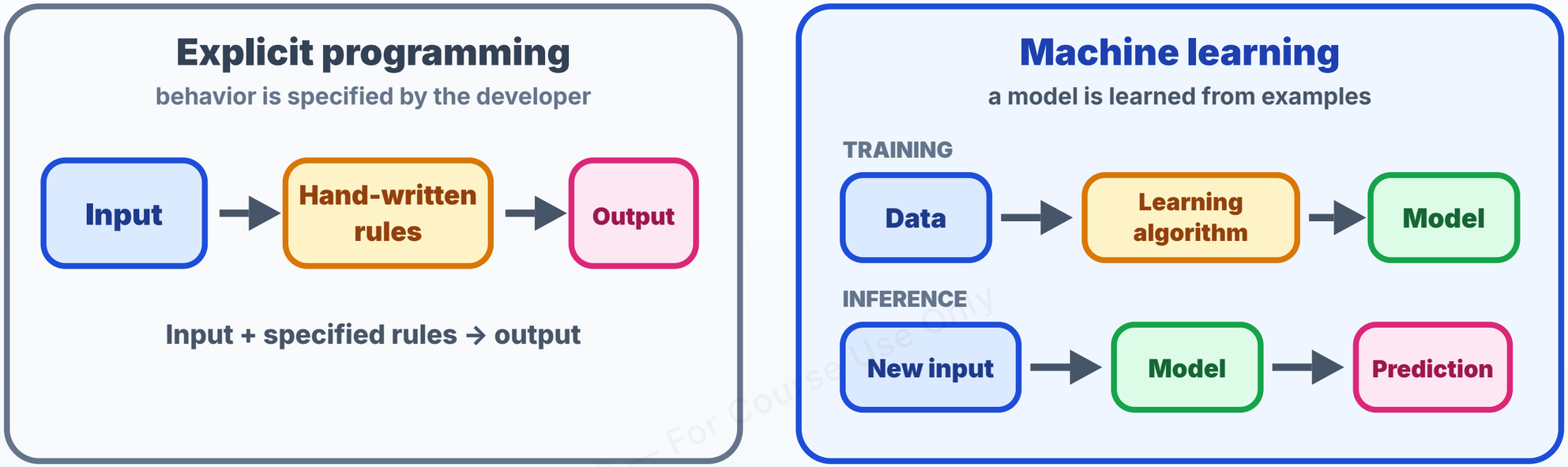
Machine learning studies methods that improve their performance by extracting **useful regularities from data or experience**.

FCIS — For Course Use Only

From Explicit Programming to Learning

For Course Only

Two ways to build a system



FCIS – For Course Use Only

💡 **Key idea.** Explicit programming specifies the behavior directly. Machine learning uses examples to **learn a model**, which is then applied to **new inputs**.

A Practical Definition of Learning

FCIS — For Educational Use Only

Task, experience, and performance

A practical description of **machine learning** involves three elements:

> A system learns from **experience** E with respect to a **task** T when its **performance** P on that task improves with experience.

Example: Email spam detection

Element	Meaning in this example
Task (T)	Classify emails as **spam or not spam**
Experience (E)	Previously **labeled email messages**
Performance (P)	Prediction quality on **unseen emails**

FCIS - For Course Use Only

> **Important.** The performance measure depends on the application. **High accuracy** does not always mean the model performs well on the errors that matter most.

When Is Machine Learning Useful?

FCIS — Classroom Use Only

Learning patterns from data

Machine learning is useful when a problem is difficult to solve using **explicit rules**, but informative examples are available.

Situation	Why learning helps	Example
Rules are difficult to specify	Patterns are easier to learn than encode manually	Fraud detection
Conditions change over time	Models can be updated using new data	Demand forecasting
Many variables interact	Models can learn relationships from examples	House-price prediction
Personalization is needed	Predictions adapt to individual preferences	Recommendations
Hidden structure exists	Algorithms can discover patterns without labels	Customer segmentation

For Course Use Only

Important. Machine learning is not always necessary. When rules are **exact, stable, and easy to specify**, conventional programming may be simpler and more reliable.

Machine Learning Is More Than Prediction

For Course Use Only

Different problems require different objectives

Machine learning can be used to **predict outcomes, discover patterns, learn representations, or choose actions**.

Learning objective	Typical question	Example output
Predict a number	What value should we expect?	House price: 850,000 SAR
Predict a category	Which class does this belong to?	Fraud / legitimate
Discover groups	Which observations are similar?	Customer segments
Learn representations	Can the data be summarized with fewer variables?	Principal components
Choose actions	Which actions lead to better long-term outcomes?	Robot navigation policy

FCIS — For Course Use Only

💡 **Key idea.** The **learning objective**, the **expected output**, and the **available feedback** help determine which machine-learning approach is appropriate.

Learning Paradigms

FCIS — Course Use Only

Different forms of feedback

The three main learning paradigms differ primarily in **what information guides learning**.

Paradigm	Available information	Main objective	Example
Supervised learning	Inputs with known targets	Predict outputs for new inputs	Loan default prediction
Unsupervised learning	Inputs without target labels	Discover patterns or structure	Customer segmentation
Reinforcement learning	Interactions and rewards	Learn actions that maximize cumulative reward	Robot navigation

FCIS — For Course Use Only

💡 **Key idea. Supervised learning** uses labeled examples, **unsupervised learning** discovers structure without target labels, and **reinforcement learning** learns through interaction and rewards.

Supervised Learning

FCIS — Private Use Only

Learning from input–output examples

In **supervised learning**, each training example contains:

$$\boxed{(x_i, y_i)}$$

where:

- x_i represents the input features.
- y_i represents the known output or **target**.

The model learns a function:

$$\hat{y} = h_\theta(x)$$

FCIS — For Course Use Only

Its goal is to produce **accurate predictions on new observations**, not merely reproduce the training examples.

> **Example: Loan default prediction.** Using customer information and historical default labels, a model learns to predict whether a new customer is likely to default.

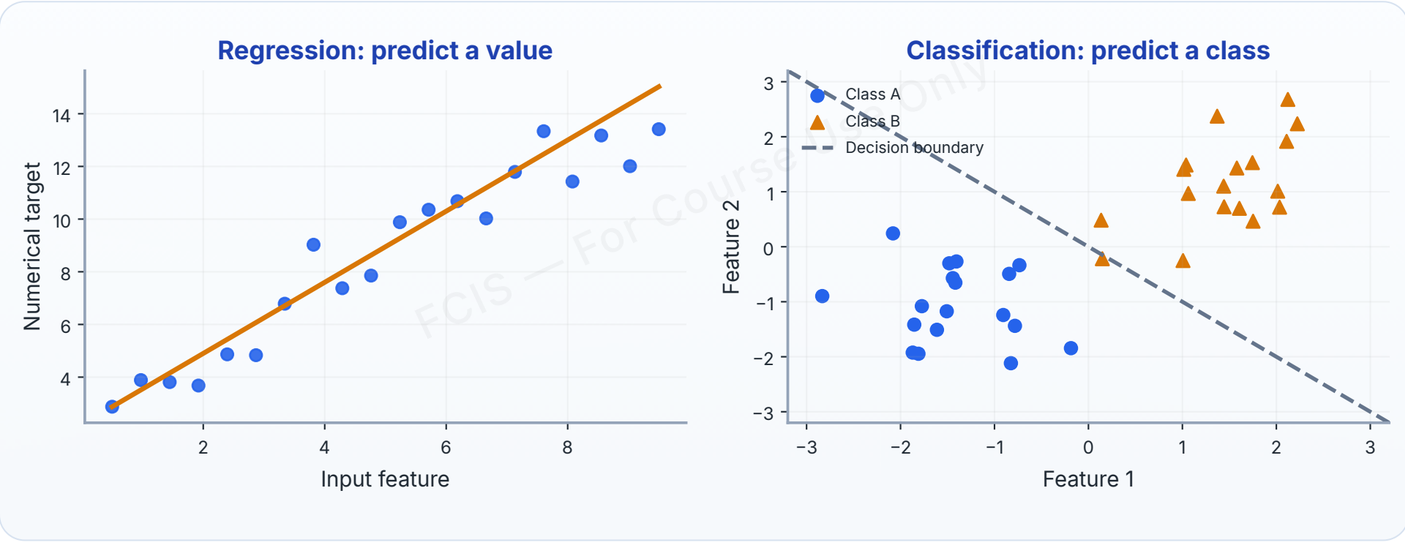
Regression vs. Classification

FCIS — For Course Use Only

Two supervised-learning tasks

The main difference is the **type of target being predicted**.

Task	Target type	Example
Regression	Numerical value	Predict apartment rent
Classification	Category or class	Predict fraud / legitimate



Important. The distinction depends on the **target variable**, not the input features. Both tasks can use numerical and categorical inputs.

Unsupervised Learning

FCIS — For Course Use Only

Discovering patterns without target labels

In **unsupervised learning**, the dataset contains input observations but no predefined target labels:

$$D = \{x_i\}_{i=1}^{n}$$

Instead of predicting known answers, the goal is to discover **useful structure** in the data.

Common objectives

- **Clustering:** Group observations with similar characteristics.
- **Dimensionality reduction:** Represent data using fewer variables.
- **Structure discovery:** Identify patterns or relationships not explicitly labeled.

FCIS - For Course Use Only

Example: Customer segmentation. A retailer groups customers based on **spending and purchasing behavior** without defining the groups in advance.

Reinforcement Learning

FCIS — For Reference Use Only

Learning through interaction and consequences

A reinforcement-learning agent repeatedly interacts with its environment:

State
Current situation

→

Action
Choice made by the agent

→

Reward
Numerical feedback

→

New State
Updated situation after acting

The objective is to learn a strategy that maximizes **cumulative reward**, not only the reward of the next action.

Element	Meaning
State	Current situation available to the agent
Action	Choice made by the agent
Reward	Numerical feedback after an action
Policy	Rule for selecting actions

FCIS – For Course Use Only

💡 This course uses **reinforcement learning** mainly to complete the conceptual map of learning paradigms. The core algorithms studied later focus on **supervised** and **unsupervised learning**.

Learning-Setting Activity

FCIS — For Educational Use Only

Identify the paradigm and task type

For each scenario, identify the **learning paradigm** and, where applicable, the **specific task**.

Scenario	Paradigm	Task type
Predict monthly electricity consumption	?	?
Discover groups of patients with similar measurements	?	?
Predict whether a transaction is fraudulent	?	?
A robot learns actions from navigation rewards	?	?
Reduce 100 measurements to a few components	?	?

FCIS — For Course Use Only

Identify the paradigm and task type

For each scenario, identify the **learning paradigm** and, where applicable, the **specific task**.

Scenario	Paradigm	Task type
Predict monthly electricity consumption	?	?
Discover groups of patients with similar measurements	?	?
Predict whether a transaction is fraudulent	?	?
A robot learns actions from navigation rewards	?	?
Reduce 100 measurements to a few components	?	?

FCIS — For Course Use Only

Suggested answers:

1. Supervised — **regression**
2. Unsupervised — **clustering**
3. Supervised — **classification**
4. **Reinforcement learning**
5. Unsupervised — **dimensionality reduction**

Part II

FCIS — For Private Use Only

Data Representation and the Learning Problem

Before an algorithm can learn, the real-world problem must be converted into **examples, features, targets, and a model representation**.

FCIS — For Course Use Only

From Real-World Objects to Data

FCIS — For Educational Use Only

A Dataset Is an Abstraction of the Problem

Suppose we want to **predict apartment prices** using the following dataset:

Apartment	Area (m^2)	Rooms	Age	Neighborhood	Price (SAR)
A	120	3	5	Central	720,000
B	90	2	12	North	510,000
C	160	4	2	Central	980,000
D	105	3	8	East	640,000

For Course Use Only

Understanding the representation

- **Observations:** Each row represents one apartment.
- **Features:** Area, rooms, age, and neighborhood.
- **Target:** `Price` is the value we want to predict.
- **Identifier:** `Apartment` distinguishes records but is not normally a predictive feature.

💡 **Key idea.** A dataset is a **simplified representation of reality**. The model can learn only from the **information included in its features**; omitted information cannot directly contribute to its predictions.

Observations, Features, and Targets

FCIS - For Course Use Only

Core Vocabulary

Using the **apartment-price dataset**, we can identify the main terms used in supervised learning:

Term	Meaning	Apartment example
Observation / sample	One **record or example** in the dataset	One apartment
Feature / predictor	Input variable used for prediction	Area, rooms, age, neighborhood
Target / response	Value or class we want to **predict**	Price
Feature vector x_i	Feature values for **one observation**	$[120, 3, 5, \text{Central}]$
Dataset size n	Total number of **observations**	$n = 4$ apartments
Number of features p	Number of **input variables**	$p = 4$ features

IDIS — For course use only

💡 **Key idea.** In supervised learning, the **target** is known during training but is **unknown for a new observation** when the model makes its prediction.

Matrix Representation

FCIS — Private Use Only

Machine-Learning Notation

For n **observations** and p **features**, the numerical feature matrix is:

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix} \in \mathbb{R}^{n \times p}$$

Here, x_{ij} is the value of **feature** j for **observation** i. Categorical features must be numerically encoded for this matrix representation.

For **supervised learning**, the targets form a vector:

$$y = [y_1, y_2, \ldots, y_n]^\top$$

💡 **Course convention: Rows** represent observations, **columns** represent features, and y contains one target per observation.

FCIS — For Course Use Only

Numerical and Categorical Features

For Course Use Only

Real datasets contain different data types

Feature type	Example	Typical representation issue
Continuous numerical	Area, temperature	Scale and distribution
Discrete numerical	Number of orders	Range and interpretation
Nominal categorical	Neighborhood	Must not invent an artificial order
Ordinal categorical	Low / medium / high	Ordered categories may carry rank information
Binary	Has loan? yes/no	Can often be represented by 0/1

FCIS — For Course Use Only

If `Neighborhood = {Central, North, East}`, coding them as 1, 2, 3 may incorrectly imply that East is “three times” Central or that distances between categories are meaningful.

💡 Lecture 2 studies encoding, scaling, missing data, transformations, and feature engineering in detail.

Feature Space

FCIS — Lecturer Use Only

Each observation occupies a point in a representation space

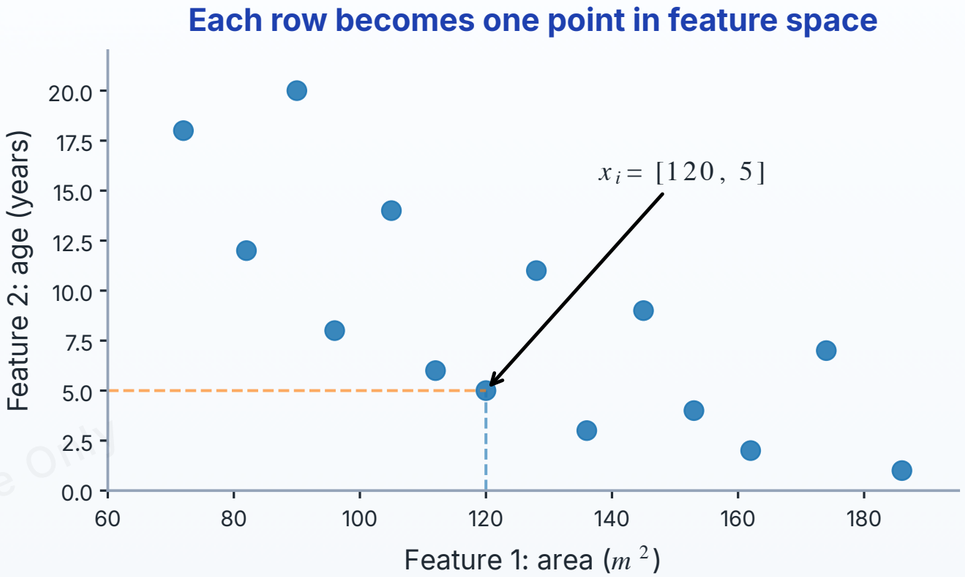
If two numerical features are used,

$$x_i = [x_{i1}, x_{i2}]$$

then each observation can be viewed as a point in a two-dimensional **feature space**.

With (p) features, the same idea extends to

$$x_i \in \mathbb{R}^p.$$



Why this matters

- **KNN** uses distance between points.
- Linear models learn boundaries or surfaces in this space.
- **K-Means** groups points around centroids.
- **PCA** changes the coordinate system to capture major directions of variation.

> How we represent a problem directly affects the **geometry** and information available to the learning algorithm.

FCIS — For Course Use Only

Representation Matters

FCIS — Private Use Only

Same real-world context, different learning problems

Consider **customer purchase history**. Different representations and objectives lead to different learning tasks.

Data representation	Learning objective
Purchase features + future spending target	**Predict future spending** (regression)
Purchase features + `Churn` target	**Predict customer churn** (classification)
Purchase features without target labels	**Discover customer segments** (clustering)
Customer states, actions, and rewards	**Learn a retention policy** (reinforcement learning)

Why does representation matter?

A poor representation may:

- **Hide useful information** about the problem.
- Introduce **misleading patterns** into the data.
- Make learning **more difficult than necessary**.

FCIS — For Course Use Only

💡 **Key idea.** A model does not observe the complete real-world situation. It learns from the **features and feedback we provide**, so representation choices strongly influence what it can learn.

The Model or Hypothesis

FCIS — Faculty Use Only

Mapping features to predictions

A **supervised model** maps input features to a predicted output:

$$\boxed{h_\theta : \mathcal{X} \to \mathcal{Y}}$$

Here, θ represents the model's **learnable parameters**.

Example: Linear regression

$$h_\theta(x) = w^\top x + b$$

FCIS — For Course Use Only

where:

- w: Feature weights.
- b: Intercept or bias.
- $\theta = (w, b)$: Collection of model parameters.

> A **model family** defines the possible functions a model can represent. **Learning** selects the parameter values that best fit the training data according to a chosen objective.

Parameters vs. Hyperparameters

FCIS — For Private Use Only

Both affect the model, but they are chosen differently

Concept	How it is determined	Examples
Parameter	Learned from **training data**	Regression coefficients, tree split thresholds, neural-network weights
Hyperparameter	Set before fitting, often selected through **validation**	Tree depth, number of neighbors k, regularization strength

Example: KNN. The algorithm stores training examples, while the number of neighbors k is a hyperparameter. We might compare $k = 3, 5, 7$ using validation data and select the value that performs best.

Important. Hyperparameters should not be selected using the **final test set**. Repeatedly tuning based on test results compromises its independence.

FCIS — For Course Use Only

What Does the Learning Algorithm Do?

FCIS — ML Course

Using data to select a model

For **supervised learning**, the training dataset is:

$$D = \{(x_i, y_i)\}_{i=1}^{n}$$

A **loss function** measures the disagreement between a prediction and its target:

$$L(h_\theta(x_i), y_i)$$

The **average training loss** is:

$$\widehat{R}(\theta) = \frac{1}{n} \sum_{i=1}^{n} L(h_\theta(x_i), y_i)$$

FCIS — For Course Use Only

A common learning objective is to find parameters that minimize this loss:

$$\theta^* = \arg\min_{\theta} \widehat{R}(\theta)$$

💡 **Key idea.** Learning aims to minimize an appropriate training objective, but the real goal is **generalization**: good performance on **unseen data**.

Loss Functions Depend on the Task

FCIS — Personal Use Only

Defining what counts as an error

Different learning problems require different **loss functions or optimization objectives**.

Learning task	Predicted output	Possible loss or objective
Regression	Numerical value	Squared error $(y - \hat{y})^2$
Binary classification	Class probability	Binary cross-entropy (log loss)
Multiclass classification	Class probabilities	Cross-entropy
Clustering	Cluster assignments	Within-cluster sum of squared distances

IUC — For Course Use Only

A **loss function** measures the cost of an individual prediction or error. A **training objective** is the quantity an algorithm minimizes, while an **evaluation metric** is used to assess model performance.

💡 **Key idea.** The quantity optimized during training and the metric used to **evaluate the final model** may be related, but they are not necessarily identical.

Training vs. Inference

FCIS — For Course Use Only

Two phases of a machine-learning system

Training

$$D_{\text{train}} \longrightarrow \text{Algorithm} \longrightarrow \hat{\theta}$$

The learning algorithm uses training examples to **fit a model**.

Key decisions:

- **Model family**
- **Loss or training objective**
- **Hyperparameters**
- **Feature preparation**

Inference

$$x_{\text{new}} \xrightarrow{h_{\hat{\theta}}} \hat{y}$$

The fitted model produces a **prediction for a new observation**.

Key requirements:

- Apply the **same preprocessing**
- Use the **learned model**
- Produce predictions **without knowing the target**

FCIS — For Course Use Only

Example: Fraud detection. During **training**, a classifier learns from historical labeled transactions. During **inference**, it predicts whether a new transaction is fraudulent.

Model Assumptions

FCIS — Private Use Only

Every model simplifies reality

A model makes learning manageable by imposing **structure** on the problem.

Examples include:

- **Linearity:** Relationships can be approximated by a linear function.
- **Local similarity:** Nearby observations often have similar outputs.
- **Rule-based structure:** Decision-tree splits can separate useful groups.
- **Representative data:** Training examples are relevant to future cases.
- **Informative features:** Available inputs contain useful signals about the target.

A **model assumption** is a condition or structural expectation about the data or relationships that supports how a model learns and makes predictions.

Important. Assumptions are often approximations rather than exact descriptions of reality. What matters is whether they are **reasonable enough for reliable predictions**.

Inductive Bias

FCIS — Academic Use Only

Learning requires preferences among possible models

A finite training dataset can be explained by **many different functions**.

To make predictions about unseen examples, a learning algorithm needs an **inductive bias**: a preference for certain solutions over others.

Examples

Model or technique	Inductive bias
Linear regression	Prefers linear relationships
KNN	Nearby observations tend to have similar outputs
Shallow decision tree	Prefers a small set of decision rules
Regularization	Favors solutions with controlled parameter complexity

FCIS — For Course Use Only

💡 **Key idea.** Inductive bias allows a model to **generalize beyond the training examples** by favoring some explanations over others.

Model Flexibility

FCIS — For Classroom Use Only

More flexible does not always mean better

Model flexibility describes how many different patterns a model can represent.

Flexibility	Potential advantage	Main risk
Low	Simple, stable, easier to interpret	**Underfitting:** misses important patterns
Moderate	Can capture useful relationships	Requires appropriate tuning
High	Can represent complex relationships	**Overfitting:** learns noise or accidental details

FCIS - For Course Use Only

Example: Decision trees. A very deep tree may classify nearly every training example correctly, yet perform **worse on new observations** than a smaller tree.

💡 **Key idea.** The goal is not to fit the training data perfectly. It is to achieve a useful balance between **learning important patterns** and **generalizing to unseen data**.

Part III

FCIS — For Private Use Only

Generalization and Model Complexity

A model is useful when it performs well on **new observations drawn from the problem we actually care about**.

FCIS — For Course Use Only

Generalization

FCIS — For Private Use Only

Performance on unseen data is the real objective

A model may perform well on its **training data** but poorly on **new observations**.

We distinguish between:

- R_{train}: Error measured on the training examples.
- R_{new}: Expected error on future, unseen observations.

Although we can measure R_{train}, our real objective is to achieve **low** R_{new}.

Generalization is the ability of a learned model to perform well on observations **not used during training**.

ECIS — For Course Use Only

💡 **Key idea.** A model should learn **patterns that transfer to new data**, not merely fit the examples it has already seen.

Why Training Error Is Optimistic

FCIS_ Private Use Only

The model has already seen the training examples

During training, the model is adjusted specifically to **reduce errors on the training data**.

As a result, **training error** is often lower than the error observed on **unseen data**.

An analogy

A student who memorizes the answers to practice questions may score perfectly on those questions but struggle with **new exam questions**.

Example: 1-Nearest Neighbor (1-NN). When evaluated on the same training observations, 1-NN typically achieves **100% training accuracy** because each observation is its own nearest neighbor. Its accuracy on **new observations** may be considerably lower.

FCIS — For Course Use Only

💡 **Key idea.** Evaluate a model using **held-out data** to obtain a more realistic estimate of its ability to generalize.

Underfitting

FCIS — For Personal Use Only

The model is too simple to capture important patterns

Underfitting occurs when a model fails to capture important relationships in the data, resulting in **poor performance even on training examples**.

Typical signs

- **High training error** and **high validation error**.
- Predictions systematically miss important patterns.
- Increasing model flexibility may improve performance.
- Better features or training procedures may also help.

FCIS — For Course Use Only

Example: Linear regression. Fitting a **straight line** to a strongly curved relationship may produce large errors on both training and validation data.

💡 **Key idea.** Underfitting often results from **insufficient flexibility**, inadequate features, or incomplete training.

Overfitting

FCIS — Personal Use Only

The model learns noise instead of generalizable patterns

Overfitting occurs when a model fits the training data too closely, capturing **noise or accidental details** that do not generalize to new observations.

Typical signs

- **Very low training error**, but noticeably **higher validation error**.
- Performance deteriorates on unseen examples.
- Predictions may be sensitive to small changes in training data.
- Increasing complexity may worsen generalization.

CIS — For Course Use Only

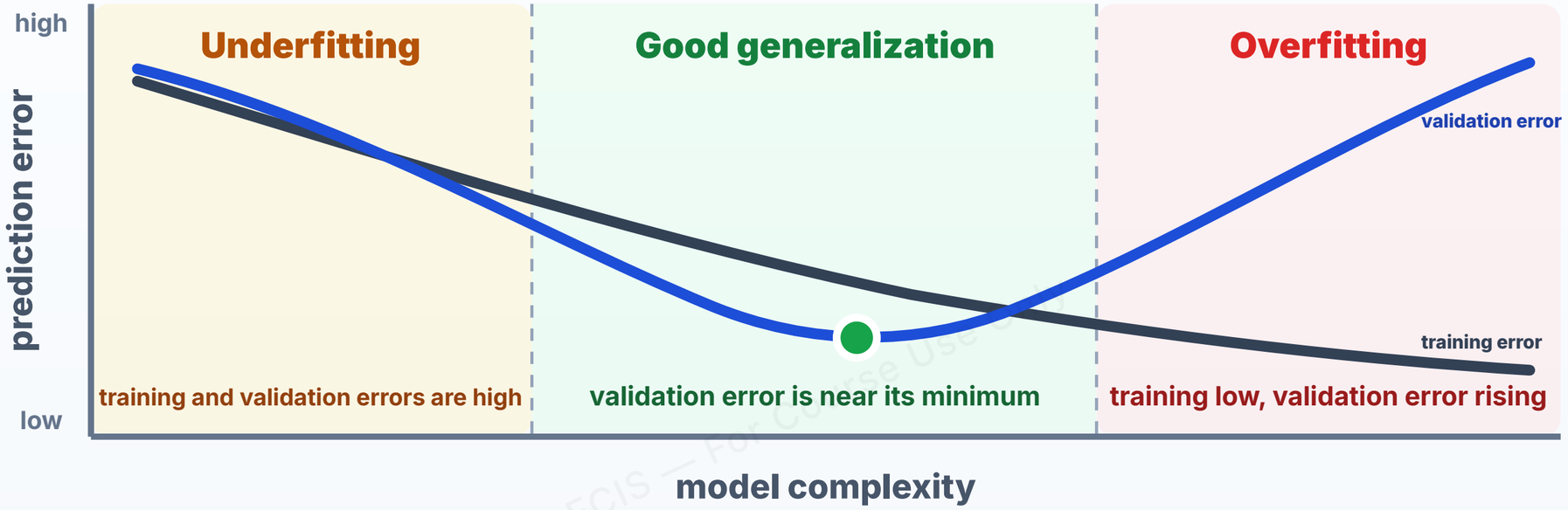
Example: Decision trees. A very deep tree may classify nearly every training observation correctly by creating **highly specific rules**, yet perform poorly on new data.

Important. Overfitting is not simply **high training accuracy**. The problem is **poor generalization despite good training performance**.

Underfitting vs. Good Fit vs. Overfitting

For Course Use Only

Training error alone is not enough



FCIS — For Course Use Only

💡 **Key idea.** As model complexity increases, **training error** usually decreases, but **validation error** may eventually rise if the model starts fitting noise. Model selection should therefore rely on **validation performance**, not training error alone.

Bias–Variance Intuition

FCIS — Course Use Only

Two sources of prediction error

High Bias

- **Restrictive assumptions** limit what the model can learn.
- Predictions **systematically miss** important patterns.
- Often associated with **underfitting**.

A **straight line** fitted to a strongly curved relationship.

High Variance

- Predictions are **sensitive to the training sample**.
- The model learns **noise and unstable details**.
- Often associated with **overfitting**.

A **very deep decision tree** trained on a small dataset.

FCIS — For Course Use Only

💡 **Key idea.** Good generalization requires balancing **model flexibility** with **prediction stability**: capturing important patterns without becoming too sensitive to the training data.

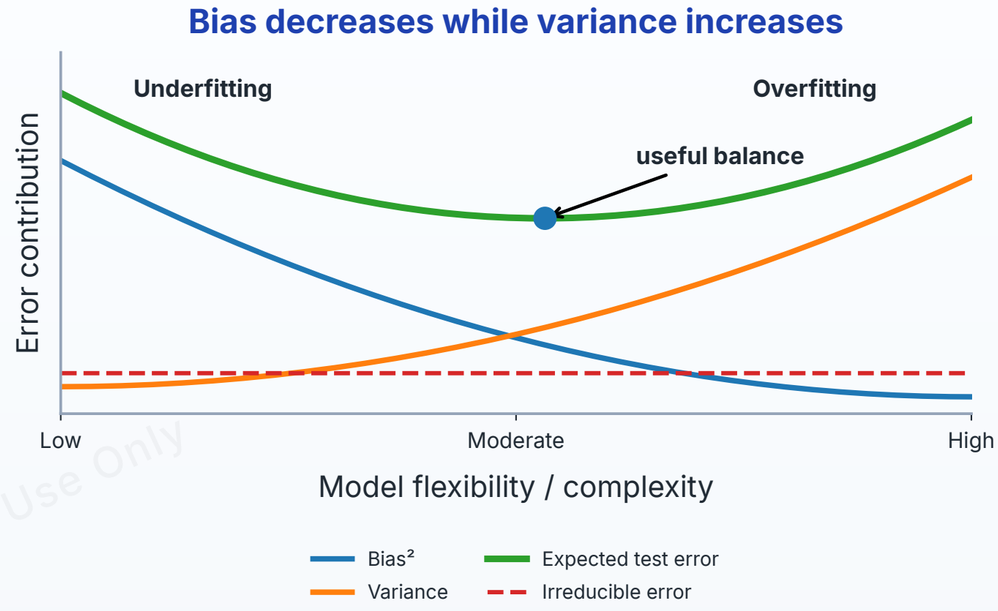
The Bias–Variance Trade-off

FCIS — For Reference Use Only

Complexity changes the kind of error we make

As model flexibility increases, a common qualitative pattern is:

- **Bias decreases** because the model can represent richer relationships.
- **Variance increases** because the fitted model can become more sensitive to the training sample.
- **Prediction error on unseen data** is often lowest at an intermediate level of flexibility.



FCIS — For Course Use Only

The goal is not to eliminate bias or variance completely. The goal is to choose a model whose **overall prediction error on unseen data is appropriately low**.

Important: this is a reasoning framework, not a rule for selecting complexity without validation data.

Noise and Irreducible Error

FCIS — For Course Use Only

Not every outcome can be predicted perfectly

Even a well-trained model may make errors because outcomes are influenced by:

- **Unobserved factors** not included in the features.
- **Measurement noise** in recorded values.
- **Random events** and unpredictable variation.

For **squared-error prediction**, a useful decomposition is:

$$\text{Expected Prediction Error} = \text{Reducible Error} + \text{Irreducible Error}$$

ISSC — For Course Use Only

Apartment prices: Two apartments with identical recorded features may sell at different prices because of **unrecorded conditions**, buyer preferences, or negotiation.

💡 **Key idea.** A more complex model cannot reliably recover **information absent from its features**. Trying to fit unpredictable noise may instead **worsen generalization**.

Part IV

FCIS — Personal Use Only

Reliable Model Selection and Evaluation

Reliable evaluation requires separating **model fitting**, **model selection**, and **final testing**, while preventing information from leaking across those roles.

FCIS — For Course Use Only

Train, Validation, and Test Sets

FCIS — Internal Use Only

Different partitions serve different purposes

A dataset is typically divided into three **non-overlapping subsets**:

Partition	Main purpose	Used for model selection?
Training set	**Fit** model parameters and structure	Not for comparing final candidates
Validation set	**Compare models** and tune hyperparameters	**Yes**
Test set	**Estimate final performance** on unseen data	**No**

For Course Use Only

The complete dataset can be written as:

$$D = D_{\text{train}} \cup D_{\text{val}} \cup D_{\text{test}}$$

where the three subsets are **disjoint**.

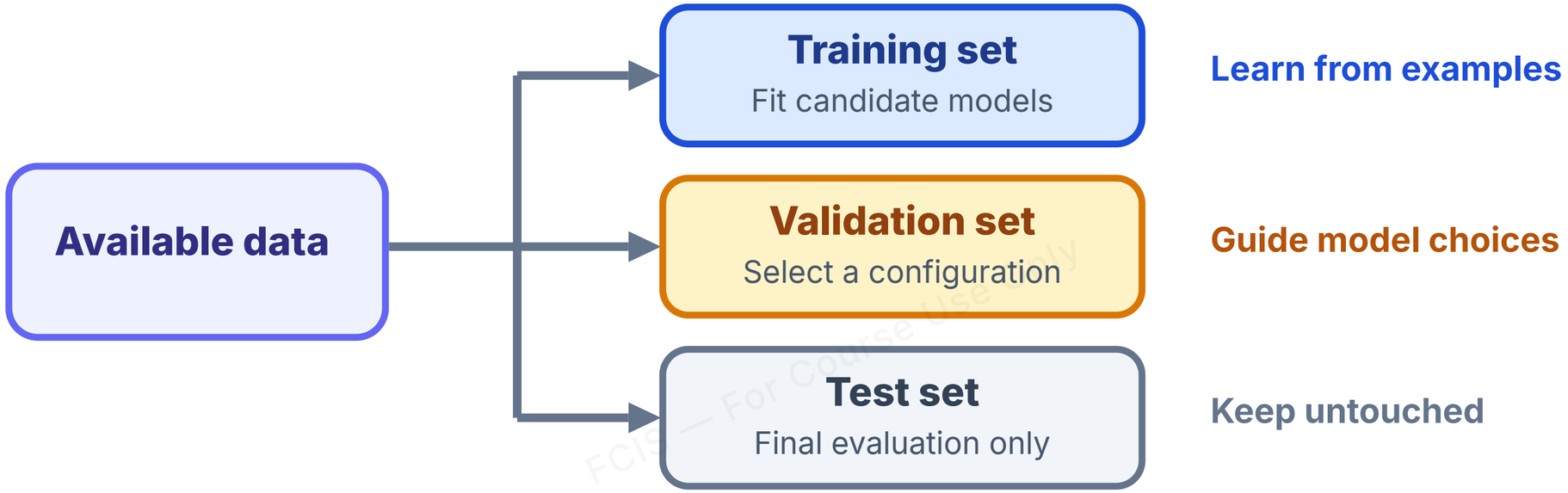
Important. There is no universal splitting ratio. The essential principle is **separating training, model selection, and final evaluation**.

Correct Model-Selection Protocol

FCIS — For Class Use Only

Keep the test set outside the design process

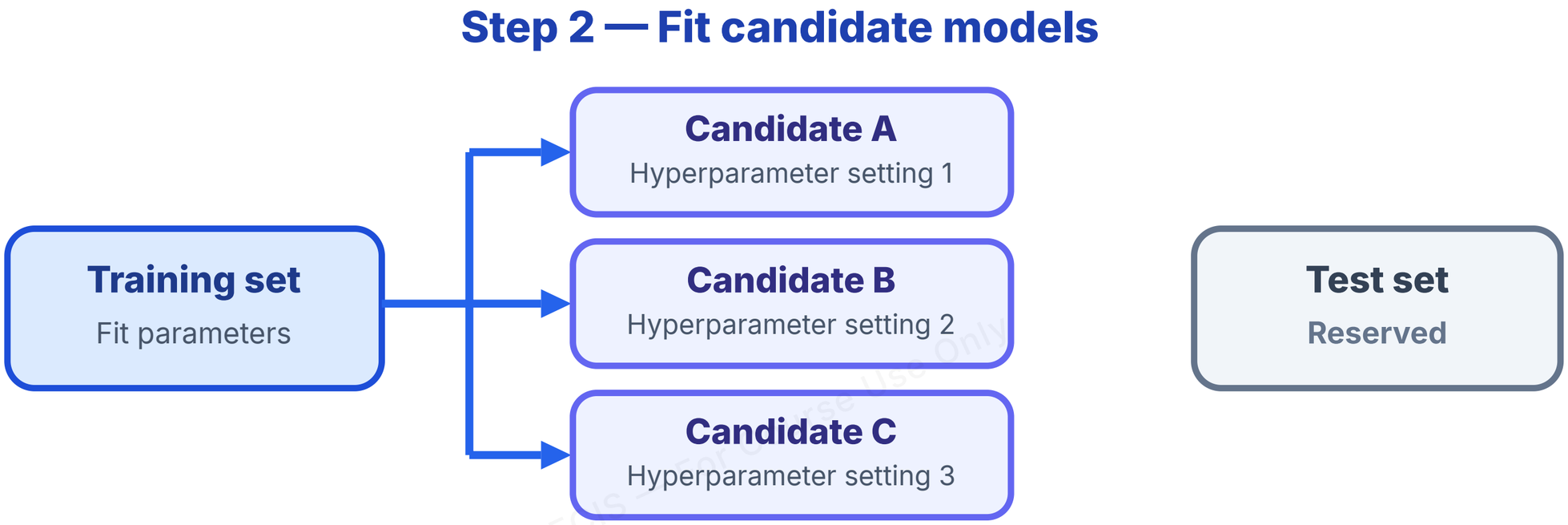
Step 1 — Split the available data



FCIS — For Course Use Only

💡 **Key idea.** Use **training data** to fit models, **validation data** to select them, and the **untouched test set** for final evaluation. **Cross-validation**, introduced later, provides another way to compare models.

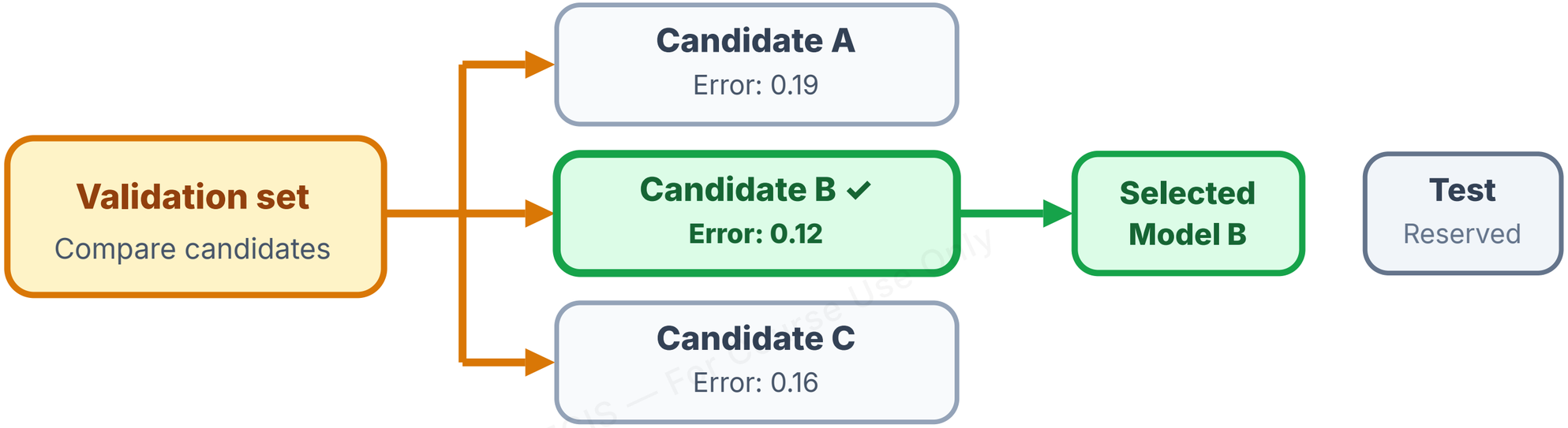
Keep the test set outside the design process



💡 **Key idea.** Use **training data** to fit models, **validation data** to select them, and the **untouched test set** for final evaluation. **Cross-validation**, introduced later, provides another way to compare models.

Keep the test set outside the design process

Step 3 — Select using validation performance

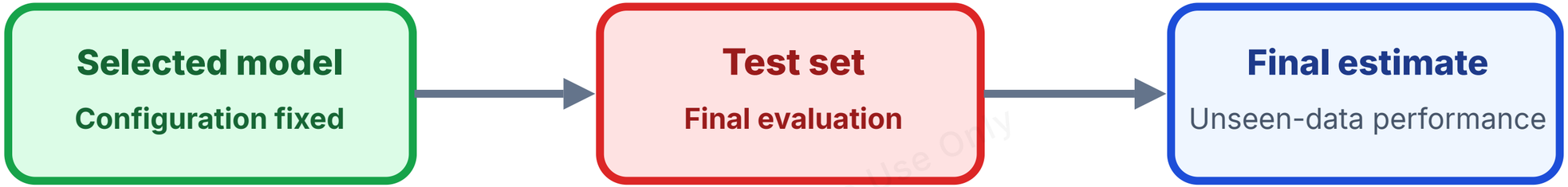


Illustrative errors: choose the model with the lowest validation error.

💡 **Key idea.** Use **training data** to fit models, **validation data** to select them, and the **untouched test set** for final evaluation. **Cross-validation**, introduced later, provides another way to compare models.

Keep the test set outside the design process

Step 4 — Evaluate on the test set



For Course Use Only

Optional: refit the fixed configuration on training + validation data before testing.

Do not tune the model after seeing test results.

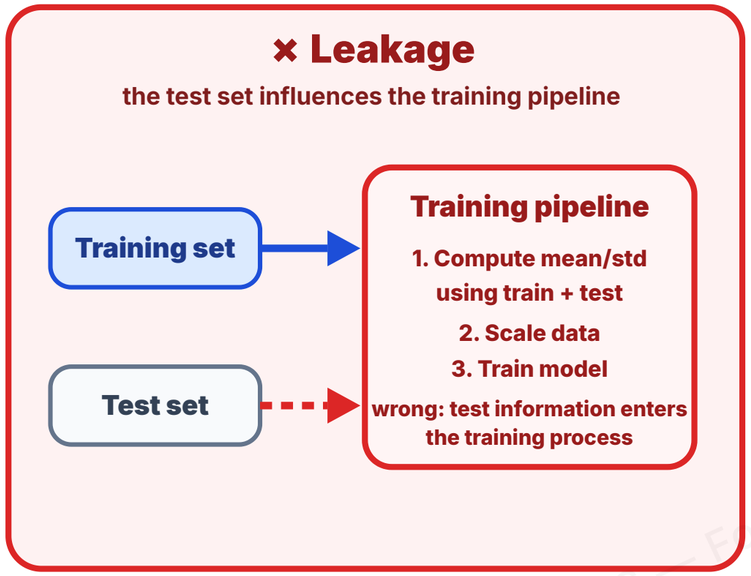
💡 **Key idea.** Use **training data** to fit models, **validation data** to select them, and the **untouched test set** for final evaluation. **Cross-validation**, introduced later, provides another way to compare models.

Data Leakage

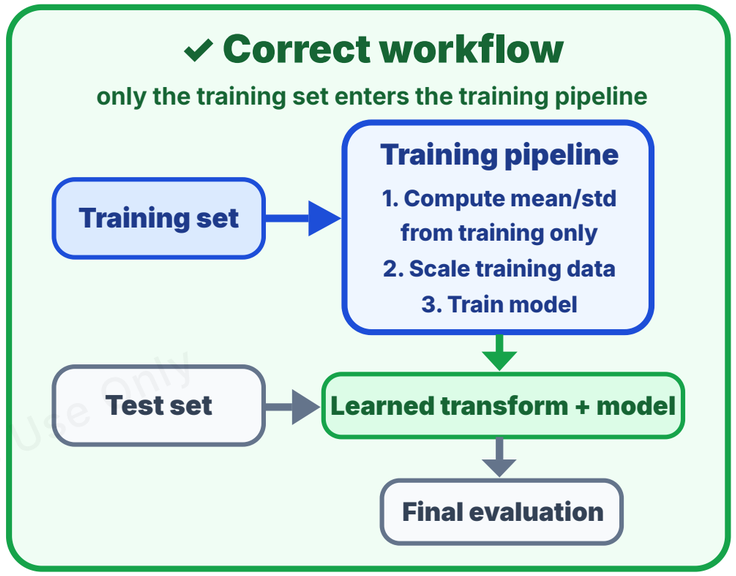
FCIS — For Course Use Only

Keep the test set outside the training pipeline

Data leakage occurs when information from outside the legitimate training process influences preprocessing, model fitting, or model selection.



Test information influences how training is performed



The test set uses the learned transform; it does not help learn it

Example. If annual income is standardized, compute μ_{train} and σ_{train} from the **training set only**, then apply the same transformation to validation and test data: $x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$

Leakage also occurs when we select features using the full dataset, tune hyperparameters after inspecting the test result, or let duplicate/future information cross the train–test boundary.

Private - For Course Use Only

Evaluation Principles

FCIS — Educational Use Only

A reliable score requires a sound evaluation design

Principle	Why it matters
Evaluate on unseen data	Estimates **generalization**, not memorization
Choose task-appropriate metrics	Different errors have **different consequences**
Use validation for model selection	Keeps the **final test set independent**
Compare against a baseline	Shows whether the model provides **meaningful improvement**
Reflect real-world conditions	Evaluation should resemble **actual deployment**
Consider uncertainty and trade-offs	A single score may **hide important weaknesses**

FCIS — For Course Use Only

💡 **Key idea.** Reliable model evaluation requires **appropriate metrics**, **independent test data**, and **realistic conditions**. Lecture 5 develops these principles using performance metrics and model-selection procedures.

Baselines

FCIS — Use Only

A model should outperform a meaningful simple reference

A **baseline** is a simple reference strategy used to determine whether a machine-learning model provides **useful improvement**.

Common baselines

Task	Simple baseline
Regression	Predict the training-set **mean or median**
Classification	Always predict the **majority class**
Time-series forecasting	Predict the **most recent observed value**

FCIS — For Course Use Only

Example: Fraud detection. If **98%** of transactions are legitimate, predicting every transaction as “legitimate” achieves **98% accuracy** while detecting **no fraudulent transactions**.

💡 **Key idea.** A model’s performance is meaningful only when compared with a **relevant baseline** and evaluated using an **appropriate metric**.

Evaluation Must Match the Real Decision

Please For Course Only

Different mistakes have different consequences

Consider a **fraud-detection system**:

Error	What happens	Possible consequence
False positive	A legitimate transaction is flagged as fraud	**Customer inconvenience** or blocked purchases
False negative	A fraudulent transaction is accepted	**Financial loss** or security risk

The importance of different errors also varies across applications:

- **Medical diagnosis:** Missing a serious condition may be more costly than requesting an additional test.
- **Loan approval:** Rejecting a reliable applicant and approving a high-risk applicant have **different consequences**.
- **House-price prediction:** An error of **10,000 SAR** may be minor for an expensive property but substantial for a cheaper one.

CIS — For Course Use Only

💡 **Key idea.** Choose evaluation **metrics and decision thresholds** that reflect the **real-world consequences** of different errors.

Worked Scenario: House-Price Prediction

For Course Usage Only

Connecting the core vocabulary

A real-estate company uses historical records containing `Area`, `Rooms`, `Age`, `Neighborhood`, and `SalePrice`.

Question	Interpretation
Learning paradigm?	Supervised learning
Task type?	Regression
One observation?	One sold property
Input features?	Area, rooms, age, neighborhood
Target variable?	Sale price
Categorical feature?	Neighborhood requires suitable encoding for many models
What does training do?	Fit a model using historical examples
What does validation do?	Select the model and hyperparameters
What does testing do?	Estimate final performance on unseen properties

Model comparison: Model A has lower training error, but Model B has lower validation error. **Model B may generalize better**, despite fitting the training examples less closely.

FCIS — For Course Use Only

Worked Scenario: Fraud Detection

FCIS — For Course Use Only

Why accuracy alone may be misleading

A fraud-detection model predicts:

$$y \in \{\text{fraud}, \text{legitimate}\}$$

Suppose the results are:

Information	Value
Training accuracy	99.0%
Validation accuracy	98.5%
Legitimate transactions in the dataset	98.7%

FCIS — For Course Use Only

Questions

1. Is **98.5% validation accuracy** necessarily good?
2. What **simple baseline** should we compare against?
3. Are **false positives and false negatives** equally costly?
4. Why must the **test set remain untouched**?

Why accuracy alone may be misleading

A fraud-detection model predicts:

$$y \in \{\text{fraud}, \text{legitimate}\}$$

Suppose the results are:

Information	Value
Training accuracy	99.0%
Validation accuracy	98.5%
Legitimate transactions in the dataset	98.7%

FCIS — For Course Use Only

Questions

1. Is **98.5% validation accuracy** necessarily good?
2. What **simple baseline** should we compare against?
3. Are **false positives and false negatives** equally costly?
4. Why must the **test set remain untouched**?

Key insight. If validation data have the same class proportions, the **majority-class baseline achieves 98.7% accuracy**, exceeding the model's 98.5%. Accuracy alone cannot tell us how well the model detects **rare fraud cases**.

Practice & Consolidation

FCIS — For Course Use Only

Reasoning About Complete Machine-Learning Problems

The goal now is to connect the lecture concepts and justify decisions about the **learning setting, representation, generalization, and evaluation**.

FCIS — For Course Use Only

Reasoning Activity

FCIS — Internal Use Only

Diagnose the learning problem

A student builds three models for the same classification task:

Model	Training Accuracy	Validation Accuracy
A	72%	70%
B	91%	88%
C	100%	76%

Answer before revealing the interpretation:

1. Which model is most likely underfitting?
2. Which model shows the strongest sign of overfitting?
3. Which model would you tentatively select?
4. Is validation accuracy alone enough for every real application?

For Course Use Only

Diagnose the learning problem

A student builds three models for the same classification task:

Model	Training Accuracy	Validation Accuracy
A	72%	70%
B	91%	88%
C	100%	76%

Answer before revealing the interpretation:

1. Which model is most likely underfitting?
2. Which model shows the strongest sign of overfitting?
3. Which model would you tentatively select?
4. Is validation accuracy alone enough for every real application?

FCIS - For Course Use Only

Suggested reasoning: **A** is the strongest underfitting candidate; **C** shows a large generalization gap; **B** is the most promising based on these values. The final decision may still depend on class balance, error costs, and the selected evaluation metric.

Common Conceptual Mistakes

FCIS — Course Use Only

Avoid These Machine-Learning Misconceptions

Misconception	Correct interpretation
“ML means neural networks.”	Neural networks are **one of many model families**.
“Unsupervised learning predicts labels.”	It discovers **structure without predefined targets**.
“More features are always better.”	Irrelevant, redundant, or leaked features can **hurt generalization**.
“100% training accuracy means an excellent model.”	Perfect training performance **does not guarantee generalization**.
“The test set is for tuning.”	Use **validation for selection** and **testing for final evaluation**.
“The most complex model is the best.”	Choose complexity based on **validation performance**, not flexibility alone.
“Accuracy is always the correct metric.”	Metrics must reflect the **task and consequences of errors**.

FCIS — For Course Use Only

Mini-Quiz

FCIS — For Course Use Only

Check your understanding

1. A system discovers groups of shoppers without predefined labels. Which learning paradigm is this?
2. In a house-price dataset, what is the difference between a feature and the target?
3. What is the difference between a parameter and a hyperparameter?
4. Why is training error usually an optimistic estimate of future error?
5. What role does the validation set play?
6. Give one example of data leakage.
7. Which pattern is more consistent with overfitting: high train/high validation error, or very low train/high validation error?

FCIS — For Course Use Only

Check your understanding

1. A system discovers groups of shoppers without predefined labels. Which learning paradigm is this?
2. In a house-price dataset, what is the difference between a feature and the target?
3. What is the difference between a parameter and a hyperparameter?
4. Why is training error usually an optimistic estimate of future error?
5. What role does the validation set play?
6. Give one example of data leakage.
7. Which pattern is more consistent with overfitting: high train/high validation error, or very low train/high validation error?

FCIS — For Course Use Only

Suggested answers: unsupervised learning; features are inputs while price is the target; parameters are learned while hyperparameters configure the learning/model; the model was optimized using the training examples; validation supports model selection; fitting preprocessing using test data is leakage; very low training error with substantially higher validation error suggests overfitting.

Lecture 1 Summary

FCIS — For Course Use Only

Key Ideas to Retain

Learning and Representation

- **Machine learning** uses data and experience to improve performance on a task.
- **Supervised learning** uses targets; **unsupervised learning** discovers structure; **reinforcement learning** uses rewards.
- **Regression** predicts numerical values; **classification** predicts categories.
- Datasets contain **observations, features, and targets**, represented using vectors and matrices.
- Models rely on **assumptions and inductive biases**; training estimates their parameters.

Generalization and Evaluation

- The goal is **performance on unseen data**, not simply low training loss.
- **Validation data** guide model and hyperparameter selection.
- **Test data** remain independent until final evaluation.
- **Underfitting** misses important patterns; **overfitting** captures noise and unstable details.
- Reliable evaluation requires **appropriate metrics, realistic conditions, and meaningful baselines**.

For Course Use Only

End-of-Lecture Check

FCIS — Private Use Only

Five questions to test your understanding

1. How do task **task** T, **experience** E, and **performance** P describe a learning system?
2. Why can the **same dataset** support a regression, classification, or unsupervised problem depending on the target and objective?
3. What is the conceptual difference between a **model parameters** and **hyperparameters**?
4. Why should **preprocessing and model selection** be performed without using information from the **test set**?
5. How can **training and validation errors** errors help distinguish **underfitting and overfitting**?

FCIS — For Course Use Only

💡 Preparation for **next lecture:** Review **numerical vs. categorical features**, summary **statistics**, **quartiles**, and the purpose of **scaling**. Lecture 2 studies **data understanding**, **preprocessing**, **transformation**, and **feature engineering** in depth.

Suggested Reading

FCIS — Private Use Only

Primary and supporting references

Primary reference

G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning with Applications in Python*, Springer, 2023.

- Chapter 1: Introduction
- Chapter 2: Statistical Learning
- Focus on the statistical learning framework, prediction vs. inference, model flexibility, training error, test error, and the bias–variance trade-off.

Supporting references

A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly, 2022.

- Review the machine-learning landscape and the end-to-end concepts of data, training, validation, and testing.

K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022.

- Use selected introductory sections for a deeper view of models, uncertainty, generalization, and probabilistic reasoning.

FCIS — For Course Use Only

💡 When reading, focus on the distinction between **fitting observed data** and **learning a pattern that performs reliably on unseen data**.